Aryan Cyrus
33114242

1. Declaration

I, Aryan Cyrus, declare that this assignment, titled FIT5032 A3, is my own original work and has not been copied from any other source except where explicitly acknowledged. I have not engaged in plagiarism, collusion, or any other form of academic misconduct in the preparation and submission of this assignment. All sources of information and data used in this assignment have been properly cited and referenced in accordance with the prescribed guidelines. I have not used unauthorized assistance in the preparation of this assignment and have not allowed any other student to copy my work. I am aware that any breach of academic integrity may result in disciplinary action as per the policies of Monash University, which may include failing this assignment or the course, and further academic penalties.

Signature: ____Aryan Cyrus_____ Date: _____15/10/2026_____

2. Github Check

Enter your Github details here.

Github Username *Enter your username here*	CYR0006-Code
Repository Shared? *Have you started and shared your assignment repository with your tutor yet?*	*CYR0006-Code/FIT5032-eFolio at A1*

3. Self-Evaluation

Rate your performance for each criteria. Put a ✅(tick) in the box where you think your work belongs.

Criteria	Exceeds Expectations	Meets Expectations	Needs Improvement	Fail to meet expectations
BR (D.1): External Authentication	✅			
BR (D.2): Email	✅			
BR (D.3): Interactive Table Data	✅			
BR (D.4): Deployment to the Cloud	✅			

BR (E.1): Cloud Functions	✅			
BR (E.2): Geo Location	✅			
BR (E.3): Accessibility	✅			
BR (E.4): Export	✅			
BR (F.1): Innovation		✅		

4. Screen Recording of BRs

Create a 6-10 minute video showing your advanced web application in action! Upload this video to your Google Drive and put the link here (ensuring that you have updated the access list so it's not private).

https://drive.google.com/file/d/1a2oKRamETT4mp790vtd9Xtbpkg5UhOsc/view?usp=sharing
Note: this video EXCLUDES BR(F) - Innovations; as per ed/rubric this is will be discussed IN CLASS and is NOT required in the video.

5. BR F.1 - Innovative Features

If you have implemented BR F.1, list your choices below and a brief description of how you implemented it.

1	Calendar and registration: Appointment Booking (using Calendar): The new Web Application must implement booking constraints such as booking conflict management using Calendar.	I implemented event booking and registration whereby the user can register for an event, and view events via a filtered calendar. This is enabled by cloud functions to manage race conditions where 2 people register at the same time.	- Adding Invite only events - Allowing the AI advisor to read calendar events and decide is a current project of the user is related
2	AI Chat: Incorporate GenAI such as Gemini API or OpenAI API or Agentic AI	I implemented an AI chat using Groq API; based on the users goal and address, the available data on the property (heat stress, available plants etc) the AI give the user personalised advice to achieve their goal.	- Implementing RAG to reduce token cost and get ONLY the info needed - Improving the System prompt to provide a more high quality response (prompt engineering)

3	Admin Dashboard:Admin Dashboard - Create a dashboard view for the system admin of this environmental charity (you may want to create a separate role for 'admin'), where they can see an overview of the website including number of users and types of users.	I implemented an admin dashboard which uses cloud functions to collect data on users (type, number, status) and show it as charts and tables using primevue tables and Chart.js	- Mapping user locations on a map that admin can interact with. - Expanding Data collection to track things like average user session time and other metrics to help improve retention.
3	Interactive charts visible to users: Interactive Charts - The new Web Application must display an interactive chart or graph using data from Firestore.	I implemented server functions to gather stats on user projects, goals and locations. I then use D3.js to visualise, chart and show this data. This also allows users to Interact with charts. For example, given a goal, users can filter all the suburbs where a certain project type is listed.	- More visualisations; for example showing cluster maps of where all user projects are - Data warehousing;One could take all the data available, create a star schema and a data warehouse. This can be used to turn data into business intelligence.

6. Reflections: Challenges

What has been the most challenging part of this assignment for you? How has this stretched you as a programmer?

I found 2 things difficult:

1) In my app, I allow users to input an address. This string is then searched against known addresses via a state API. This was a challenge because it needed to be optimised. The address search itself is very easy, but comparing the input string to thousands of records was slow. For example if I were to use 'like %x%' the prefix/suffic matching would take too long and the search API would time out. My solution was 2 things:
    a) I changed to prefix matching only
    b) Searching against structured field individually (Street name, suburb etc)

Combined, these 2 brought the search time from 4-6 seconds down to 1 second.

2) Another hard part was making event registration race-safe. A naive "check capacity, then write" has a window where two users can both pass the check and overbook the last spot. I had to learn and apply Firestore transactions properly. I had to read the capacity, the user's existing booking, and same-day conflicts, all inside one atomic runTransaction call.

7. Declaration: Additional Help

Any tools that you used (including Gen AI or existing code reuse) must be declared here.

Note: GenAI is not allowed for coding purposes in any assignment,

However, you may use GenAI for brainstorming, problem solving and learning. You need to declare all such uses here. One row per help used. More details on how to acknowledge the use of Gen AI can be found here.

Name	**Description**
Claude for brainstorming	*I used claude to brainstorm my options for user side security measures such as XSS protection and input validation.*
Claude for checking syntax	I used claude code to confirm Vue/CSS/primvue syntax where I was not sure.

8. Learning Summary

8.1 Coverage of the Learning Outcomes

LO1: History's impact on current web tech
This app is a serverless SPA. Early web apps were full page reloads per request); this app uses client-side routing (createWebHistory in src/router/index.js) so navigation never leaves the page. I thus understand how current technologies compare to legacy web applications.

LO2: Design, construct, and publish a web-database app
End-to-end: Firestore as the database, a Vue 3/Vite SPA as the client, Cloud Functions as the server tier, and a published deployment on Firebase Hosting (firebase.json, .firebaserc).

LO3: Analyse and critique key technological issues
There were concrete trade-offs I had to resolve; the prefix matching issues and data visualisation problems above are 2 examples of this.

LO4: Test key features of commonly used languages
I had to use various patterns in Vue (ref/computed/watch, async onAuthStateChanged) to account for code throws and ensure everything worked. Also exercised Firestore's in firestore.rules.

LO5: Assess and apply the MVC pattern
GirtByGreen (my app) follows the MVC pattern.

LO6: Professional approach: data privacy and cybersecurity

- Server-authoritative privilege: every actual admin action is re-checked server-side via requireAdmin() in functions/index.js, so a forged client request can't bypass it.
- Firestore security rules as a second trust
- Cloud functions: SendGrid and Groq API keys are Firebase-managed secrets (defineSecret) injected only inside Cloud Functions.
- Input sanitization in depth: Server-side sanitize-html with an explicit allowlist before anything reaches an email (the one place user-authored HTML reaches a real inbox). Also client-side containsHtml() validation.

8.2 Reflection

Most important thing I've learned:

The most important thing I have learnt is security. It quickly became clear that security has layers (input validation, server-side validation, firestore rules, roles etc). These all have jobs and come together to make a secure system.

The things that helped me the most:

1) In the early days the workshops/labs were helpful. This is ESPECIALLY because they were short and to the point; enough detail to teach concepts, but not too long such that it would affect my ability to keep up with workload or do busy work.
2) AI: AI was very helpful because I was very new to Vue. Syntax, conventions and ideation was a big part of the learning process which AI helped explain.

I found the following topics particularly challenging:

- Firestore transactions and atomicity (bookEvent's capacity/conflict check), and reasoning about Firestore security rules as a separate language with its own evaluation model.
- APIs: How they work, and to make urls and requests and so on.

I found the following topics particularly interesting:

- Combining an LLM with live external data
- Firestore transactions and atomicity
- Security

I feel I learnt these topics well:

- Vue's reactivity model (ref/computed/watch) and composables as a way to share stateful logic; I can see now how they map to React (which I used before this unit)
- Firestore transactions and atomicity

I still need to work on:

I need to improve my testing; my current project has no unit tests, no husky, no CI/CD pipeline. This is essential in a production environment and something that I need to spend more time on.

My progress in this unit was:

Steady and slow at the start, and accelerated greatly during A3 as I implemented many systems at once and learned how they work in practice.

If I did this unit again I would:
Write Firestore rules from day one; I feel like to this day firestore rules are not very familiar to me; so I do wish I had focused more on it from the start.